

Magezi Architecture Decisions

LLM Backend: Groq (Llama 3.3 70B)

Primary LLM is **Groq** running Llama 3.3 70B Versatile (free tier). Falls back to Claude Sonnet 4 if Anthropic API key is available. Falls back to structured local response if both fail.

Retry logic: On HTTP 429 (rate limit), retries 3 times with exponential backoff (2s, 4s). On all retries exhausted, the formatted local fallback kicks in — student always gets a response.

Fallback chain: Groq stream → Groq retry → Local formatted response (word-by-word SSE chunks).

Agentic Tutor Loop

Every query goes through `tutor_loop.py` before generation: 1. **Complexity detection:** simple (2 steps) / moderate (4 steps) / complex (6 steps) 2. **Calculation awareness:** detects numeric problems, injects formulas 3. **Multi-retrieval:** “Compare X and Y” splits into sub-queries, retrieves each separately 4. **Teaching plan:** injected into LLM prompt so responses follow NCDC pedagogy

STEM Tools (`tools.py`)

- **18 physical constants** (NIST CODATA): speed of light, Planck, Boltzmann, etc.
- **47 formulas** across 4 subjects: $F=ma$, $pH=-\log[H^+]$, quadratic formula, etc.
- **Safe calculator:** evaluates expressions without arbitrary code execution
- Tool context injected into every prompt + appended to local fallback responses

Knowledge Base (4,457 passages)

- **Source:** 4 official NCDC 2025 syllabus PDFs (288 pages)
- **Extraction:** `indexer.py` uses PyMuPDF, chunks ~160 words with overlap
- **Structured data:** 4 JSON files with topics, subtopics, competences (68 entries)
- **Retrieval:** Keyword search with bigram boosting + section title matching
- **Multilingual bridge:** 120 Luganda/Swahili/Runyankole terms → English

SSE Streaming + Fallback

- Groq tokens stream via SSE `data: events`
- If Groq fails (429/403), `_chunk_fallback()` yields the formatted local response word-by-word — frontend renders identically
- Faithfulness: LLM responses scored by token overlap; local fallback trusted at 0.95
- Multi-turn context: `_save_turn()` called after stream completes; 5-turn sliding window

Frontend Chat UX (Grok-Inspired)

- **Markdown rendering:** react-markdown + remark-gfm + normaliseMarkdown() preprocessor
- **Action buttons:** Copy (clipboard API + checkmark), Regenerate (resends last query), Listen (Web Speech TTS), Thumbs up/down (TanStack mutation)
- **Scroll-to-bottom:** Floating button appears when >200px from bottom
- **Expand/collapse:** Messages >1500 chars get toggle with gradient fade
- **Citations auto-expanded:** `<details open>` shows sources immediately
- **Quick suggestions:** Context-aware chips — starters before first message, follow-ups after
- **Mobile-first:** 100dvh, 32px avatars on mobile, sticky composer with safe-area padding

State Management

Concern	Tool
Chat, locale, subject, autoNarrate	Zustand (persist to localStorage)
Auth token + user profile	Zustand (separate store, persist)
Health polling, feedback, subjects	TanStack Query

Security

- CORS: configurable `ALLOWED_ORIGINS` (or `*` for dev)
- Rate limiter: thread-safe, sliding window, 429 response, stale IP eviction
- Session: thread-safe, 24h TTL, 5K max, deque(maxlen=20)
- Auth: bcrypt + JWT (72h) + BYOK

- Input guard: OWASP LLM01 + exam cheating block
- Passage spotlight: hash-bound markers prevent indirect injection

Tech Stack

- **Frontend:** Next.js 16, React 19, TypeScript 5.8, Zustand 5, TanStack Query 5, react-markdown
- **Backend:** FastAPI, Python 3.11+, Groq API (OpenAI-compatible)
- **Knowledge Base:** 4 NCDC PDFs (PyMuPDF) + 4 JSON syllabus files
- **Deployment:** Docker Compose (Redis + Qdrant + FastAPI + Next.js)